VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum - 590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

ARUN DURGANNAVAR(1BM24CS054)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B. M. S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU - 560019

February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore - 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by Arun Durgannavar(1BM24CS054), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Syed Akram	**Dr. Kavitha Sooda**
Assistant Professor	Professor and Head
Department of CSE	Department of CSE
BMSCE, Bengaluru	BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab Program 1: Topological Sorting

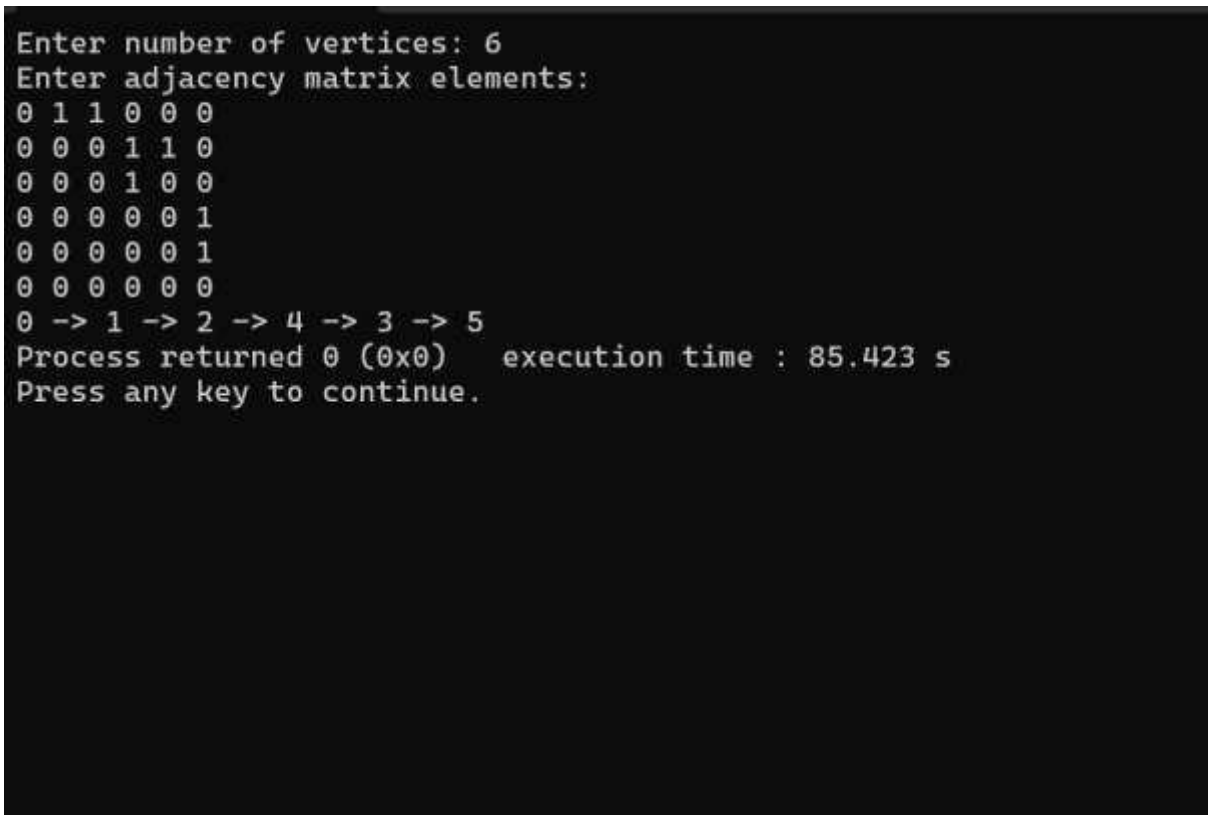
Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
#define MAX 100

void topological(int adj[MAX][MAX], int n) {
    int indeg[MAX] = {0};
    int topo[MAX];
    int queue[MAX];
    int front = 0, rear = -1;
    int count = 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (adj[i][j] == 1)
                indeg[j]++;
        }
    }

    for (int i = 0; i < n; i++) {
        if (indeg[i] == 0)
            queue[++rear] = i;
    }

    while (front <= rear) {
        int u = queue[front++];
        topo[count++] = u;
```

```
        for (int j = 0; j < n; j++) {
            if (adj[u][j] == 1) {
                indeg[j]--;
                if (indeg[j] == 0)
                    queue[++rear] = j;
            }
        }
    }

    if (count != n)
        printf("Topological ordering not possible (Graph has cycle)\n");
    else {
        for (int i = 0; i < count; i++) {
            printf("%d", topo[i]);
            if (i != count - 1)
                printf(" -> ");
        }
    }
}

int main() {
    int n;
    int adj[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix elements:\n");
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    topological(adj, n);
    return 0;
}
```

Output:



Lab Program 2: Johnson-Trotter Algorithm

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>

int a[20], dir[20], n;

void print()
{
   for(int i = 0; i < n; i++)
      printf("%d ", a[i]);
   printf("\n");
}

int getMobile()
{
   int mobile = 0, pos = -1;

   for(int i = 0; i < n; i++)
   {
      if(dir[i] == -1 && i != 0 && a[i] > a[i-1] && a[i] > mobile)
      {
         mobile = a[i];
         pos = i;
      }

      if(dir[i] == 1 && i != n-1 && a[i] > a[i+1] && a[i] > mobile)
      {
         mobile = a[i];
         pos = i;
```

```
        }
    }

    return pos;
}

void generatePermutations()
{
    int mobilePos, mobile;

    print();

    while(1)
    {
        mobilePos = getMobile();

        if(mobilePos == -1)
            break;

        mobile = a[mobilePos];

        if(dir[mobilePos] == -1)
        {
            int temp = a[mobilePos];
            a[mobilePos] = a[mobilePos-1];
            a[mobilePos-1] = temp;

            temp = dir[mobilePos];
            dir[mobilePos] = dir[mobilePos-1];
            dir[mobilePos-1] = temp;
```

```
            mobilePos--;
        }
        else
        {
            int temp = a[mobilePos];
            a[mobilePos] = a[mobilePos+1];
            a[mobilePos+1] = temp;

            temp = dir[mobilePos];
            dir[mobilePos] = dir[mobilePos+1];
            dir[mobilePos+1] = temp;

            mobilePos++;
        }

        for(int i = 0; i < n; i++)
        {
            if(a[i] > mobile)
                dir[i] = -dir[i];
        }

        print();
    }
}

int main()
{
    printf("Enter value of n: ");
    scanf("%d", &n);
```

```
    for(int i = 0; i < n; i++)
    {
        a[i] = i + 1;
        dir[i] = -1;
    }

    generatePermutations();

    return 0;
}
```

Output:

```
Enter value of n: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Process returned 0 (0x0)   execution time : 2.667 s
Press any key to continue.
```

Lab Program 3: Merge Sort

Sort a given set of N integer elements using Merge Sort technique and compute its time taken.

```
#include <stdio.h>
#include <time.h>

void merge(int arr[], int low, int mid, int high)
{
   int i, j, k;
   int temp[10000];

   i = low;
   j = mid + 1;
   k = low;

   while(i <= mid && j <= high)
   {
      if(arr[i] <= arr[j])
         temp[k++] = arr[i++];
      else
         temp[k++] = arr[j++];
   }

   while(i <= mid)
      temp[k++] = arr[i++];

   while(j <= high)
      temp[k++] = arr[j++];

   for(i = low; i <= high; i++)
      arr[i] = temp[i];
}

void mergeSort(int arr[], int low, int high)
```

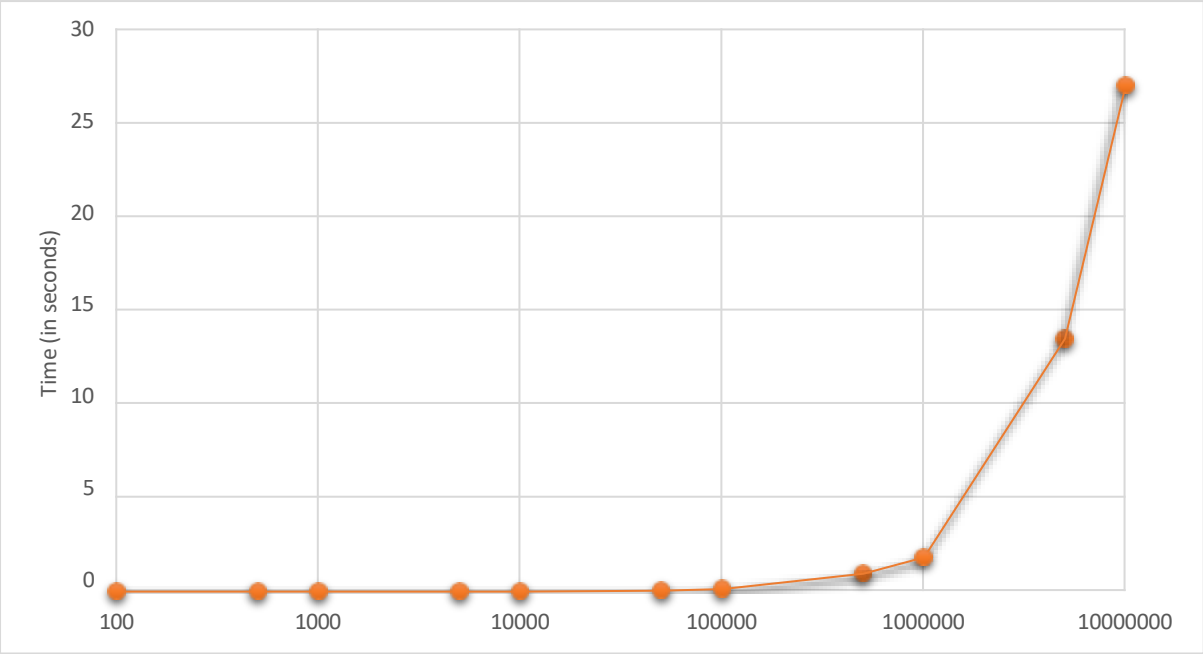
```
{
    if(low < high)
    {
        int mid = (low + high) / 2;

        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);

        merge(arr, low, mid, high);
    }
}

int main()
{
    int n;
    int arr[10000];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    clock_t start, end;

    start = clock();

    mergeSort(arr, 0, n - 1);

    end = clock();

    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
    printf("\nSorted array:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n\nExecution Time = %lf seconds\n", time_taken);


    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter elements:
38 27 43 3 9

Sorted array:
3 9 27 38 43

Execution Time = 0.000000 seconds

Process returned 0 (0x0)   execution time : 28.576 s
Press any key to continue.
```

Time Complexity Analysis:



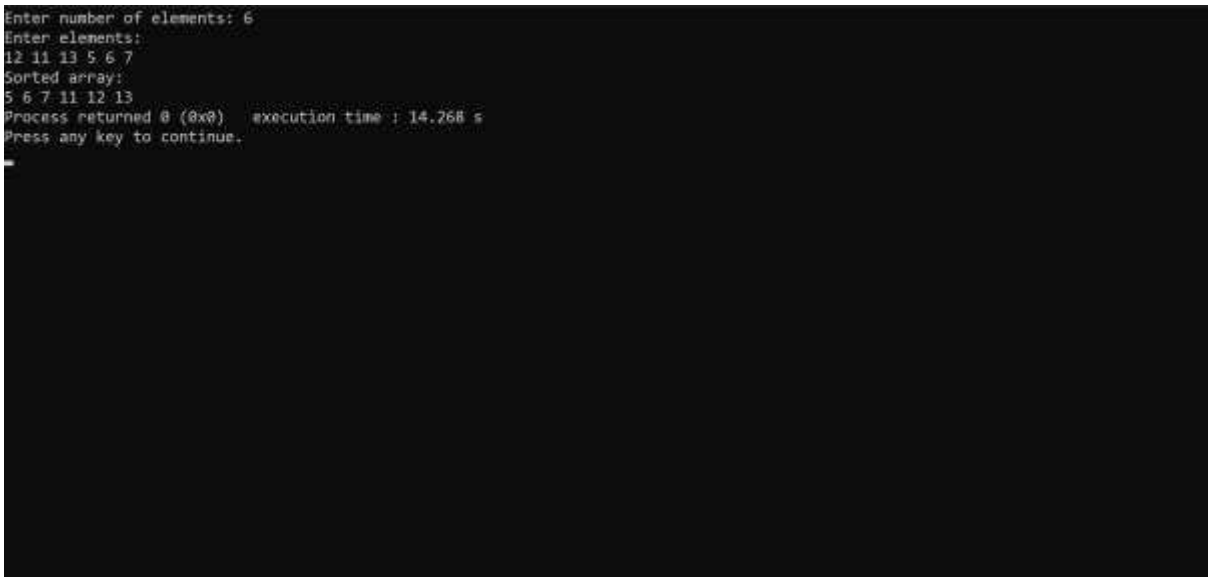
Lab Program 4: Quick Sort

```
#include <stdio.h>

int partition(int a[], int low, int high)
{
   int pivot, i, j, temp;

   pivot = a[low];
   i = low + 1;
   j = high;

   while (1)
   {
      while (i <= high && a[i] <= pivot)
         i++;

      while (a[j] > pivot)
         j--;

      if (i < j)
      {
         temp = a[i];
         a[i] = a[j];
         a[j] = temp;
      }
      else
         break;
   }

   temp = a[low];
   a[low] = a[j];
   a[j] = temp;
```

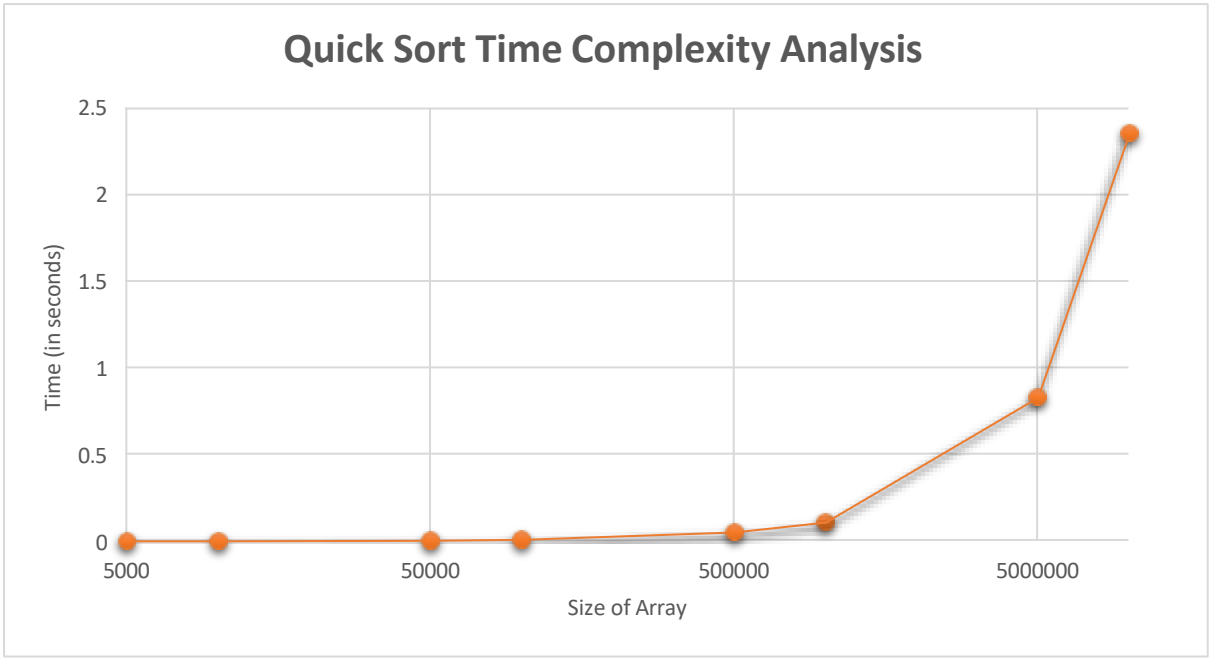
```
    return j;
}

void quickSort(int a[], int low, int high)
{
    int j;

    if (low < high)
    {
        j = partition(a, low, high);

        quickSort(a, low, j - 1);
        quickSort(a, j + 1, high);
    }
}

int main()
{
    int a[100], n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    printf("Sorted array:\n");
     for (int i = 0; i < n; i++)
         printf("%d ", a[i]);
```

```
    return 0;
}
```

Output:



Time Complexity Analysis:



Size of Array	Time(in milliseconds)
5000	0
10000	0
50000	4
100000	9
500000	51
1000000	108
5000000	831
10000000	2361

Lab Program 5: Heap Sort

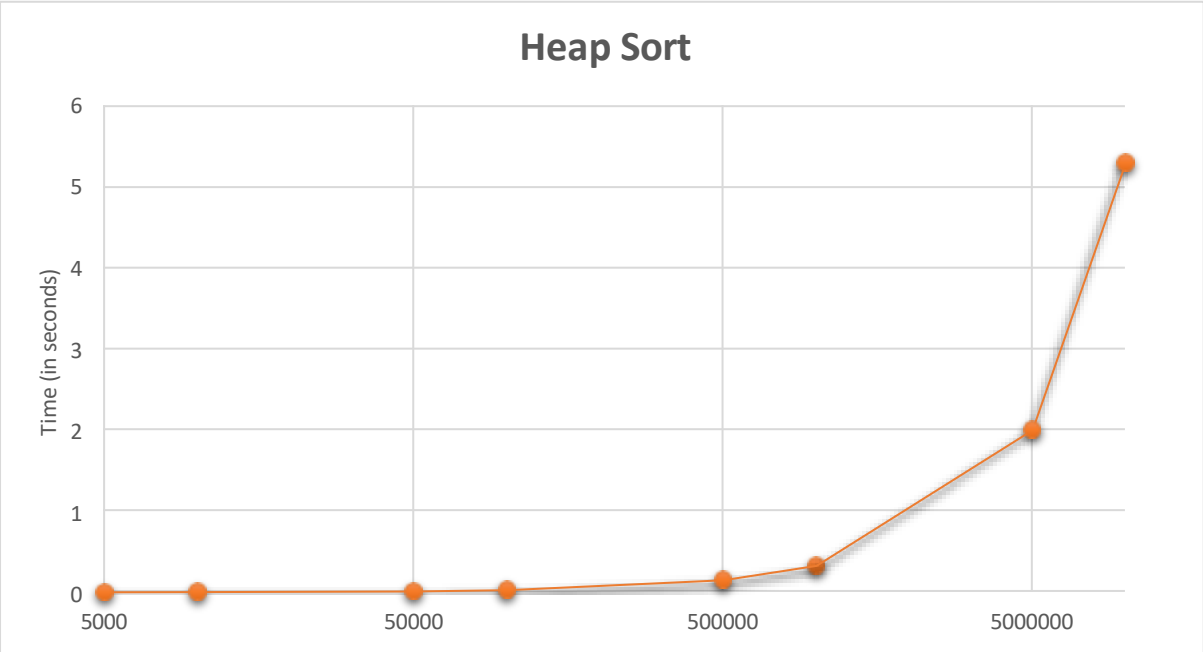
```
#include <stdio.h>
#include <time.h>

void heapify(int a[], int n, int i)
{
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;
   int temp;

   if (left < n && a[left] > a[largest])
      largest = left;


   if (right < n && a[right] > a[largest])
      largest = right;


   if (largest != i)
   {
      temp = a[i];
      a[i] = a[largest];
      a[largest] = temp;

      heapify(a, n, largest);
   }
}

void heapSort(int a[], int n)
{
   int i, temp;


   for (i = n / 2 - 1; i >= 0; i--)
   {
      heapify(a, n, i);
   }


   for (i = n - 1; i > 0; i--)
```

```
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

int main()
{
    int a[1000], n, i;
    clock_t start, end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter  elements:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    start = clock();

    heapSort(a, n);

    end = clock();

    printf("\nSorted  array:\n");

    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\n\nTime taken = %f seconds\n", time_taken);

    return 0;
}
```

Output:

```
Enter number of elements: 6
Enter elements:
12 11 13 5 6 7

Sorted array:
5 6 7 11 12 13

Time taken = 0.000000 seconds

Process returned 0 (0x0)   execution time : 32.861 s
Press any key to continue.
```

Time Complexity Analysis:



Size of Array	Time(in milliseconds)
5000	0
10000	2
50000	11
100000	26
500000	149
1000000	325
5000000	2005
10000000	5306

Lab Program 6: 0/1 Knapsack

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>


int max(int a, int b)

{

   if(a > b)

      return a;

   return b;

}


int knapsack(int W, int wt[], int val[], int n)

{

   int i, w;

   int K[n + 1][W + 1];


   for(i = 0; i <= n; i++)

   {

      for(w = 0; w <= W; w++)

      {

         if(i == 0 || w == 0)

            K[i][w] = 0;


         else if(wt[i - 1] <= w)

            K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]],

                  K[i - 1][w]);


         else
```

```
                K[i][w] = K[i - 1][w];
            }
        }

    return K[n][W];
}

int main()
{
    int n, W;

    printf("Enter number of items: ");
    scanf("%d", &n);

    int wt[n], val[n];

    printf("Enter weights:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    printf("Enter profits:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    printf("Enter capacity: ");
    scanf("%d", &W);

    printf("Maximum Profit = %d", knapsack(W, wt, val, n));
```

```
    return 0;
}
```

Output:

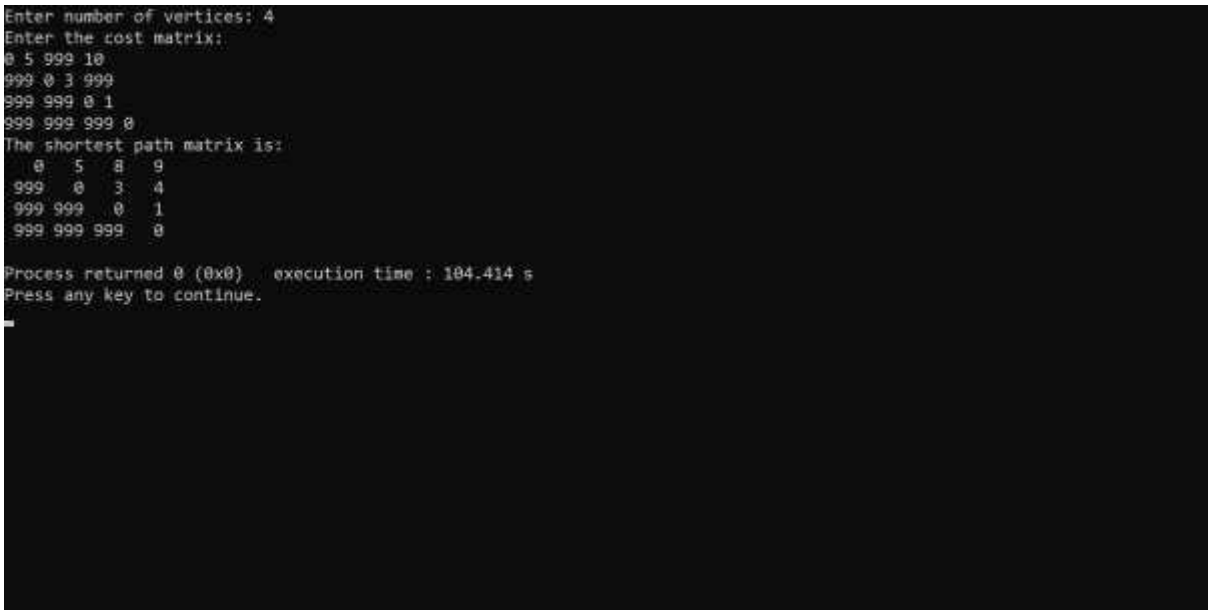
Lab Program 7: Floyd's Algorithm

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>

#define INF 999


void floyd(int n, int cost[10][10])
{
   int i, j, k;

   for(k = 0; k < n; k++)
   {
      for(i = 0; i < n; i++)
      {
         for(j = 0; j < n; j++)
         {
            if(cost[i][k] + cost[k][j] < cost[i][j])
               cost[i][j] = cost[i][k] + cost[k][j];
         }
      }
   }

   printf("The shortest path matrix is:\n");

   for(i = 0; i < n; i++)
   {
      for(j = 0; j < n; j++)
         printf("%4d", cost[i][j]);

      printf("\n");
   }
}
```

```
int main()
{
   int n, cost[10][10];

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter the cost matrix:\n");

   for(int i = 0; i < n; i++)
   {
      for(int j = 0; j < n; j++)
         scanf("%d", &cost[i][j]);
   }

   floyd(n, cost);

   return 0;
}
```

Output:



Lab Program 8(a): Prim's Algorithm

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

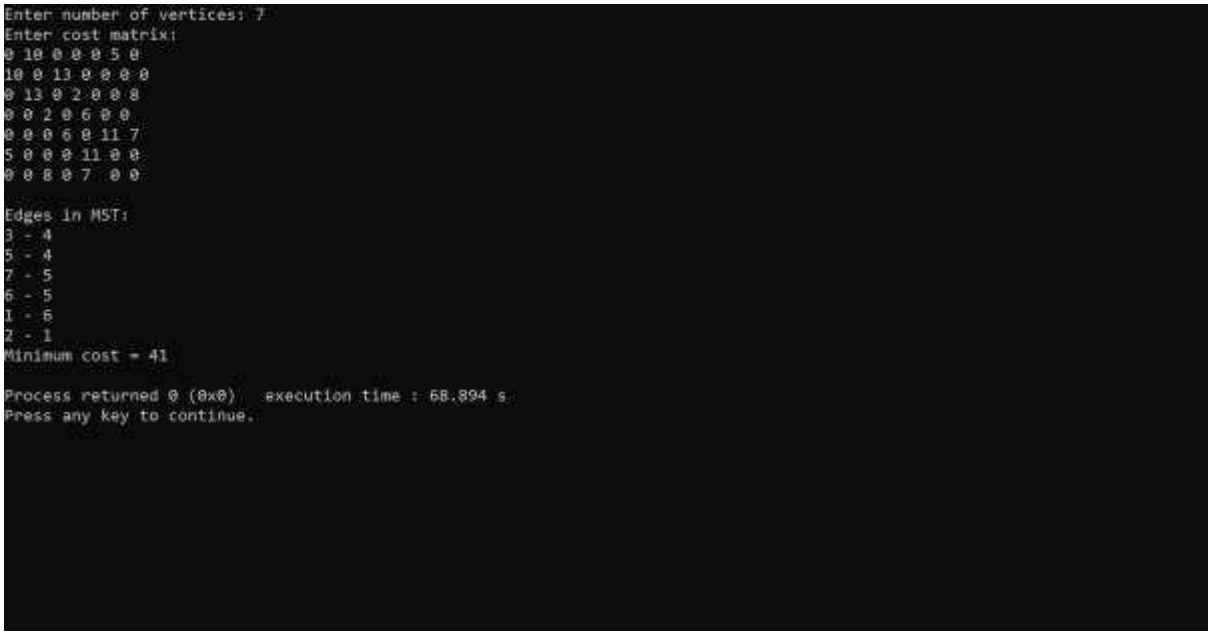
```
#include <stdio.h>
#define INF 9999
#define MAX 10

int main() {
   int n, cost[MAX][MAX];
   int near[MAX], t[MAX][2];
   int i, j, k, l, min, mincost = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter cost matrix:\n");
   for (i = 1; i <= n; i++) {
      for (j = 1; j <= n; j++) {
         scanf("%d", &cost[i][j]);
         if (cost[i][j] == 0)
            cost[i][j] = INF;
      }
   }

   min = INF;
   for (i = 1; i <= n; i++) {
      for (j = i + 1; j <= n; j++) {
```

```
        if (cost[i][j] < min) {
            min = cost[i][j];
            k = i;
            l = j;
        }
    }
}


t[1][1] = k;
t[1][2] = l;
mincost = cost[k][l];


for (i = 1; i <= n; i++) {
    if (cost[i][k] < cost[i][l])
        near[i] = k;
    else
        near[i] = l;
}

near[k] = 0;
near[l] = 0;

for (i = 2; i <= n - 1; i++) {
    min = INF;


    for (j = 1; j <= n; j++) {
```

```
        if (near[j] != 0 && cost[j][near[j]] < min) {
            min = cost[j][near[j]];
            k = j;
        }
    }


    t[i][1] = k;
    t[i][2] = near[k];
    mincost += cost[k][near[k]];

    near[k] = 0;


    for (j = 1; j <= n; j++) {
        if (near[j] != 0 && cost[j][near[j]] > cost[j][k]) {
            near[j] = k;
        }
    }
}


printf("\nEdges in MST:\n");
for (i = 1; i <= n - 1; i++) {
    printf("%d - %d\n", t[i][1], t[i][2]);
}

printf("Minimum cost = %d\n", mincost);
```

```
    return 0;
}
```

Output:



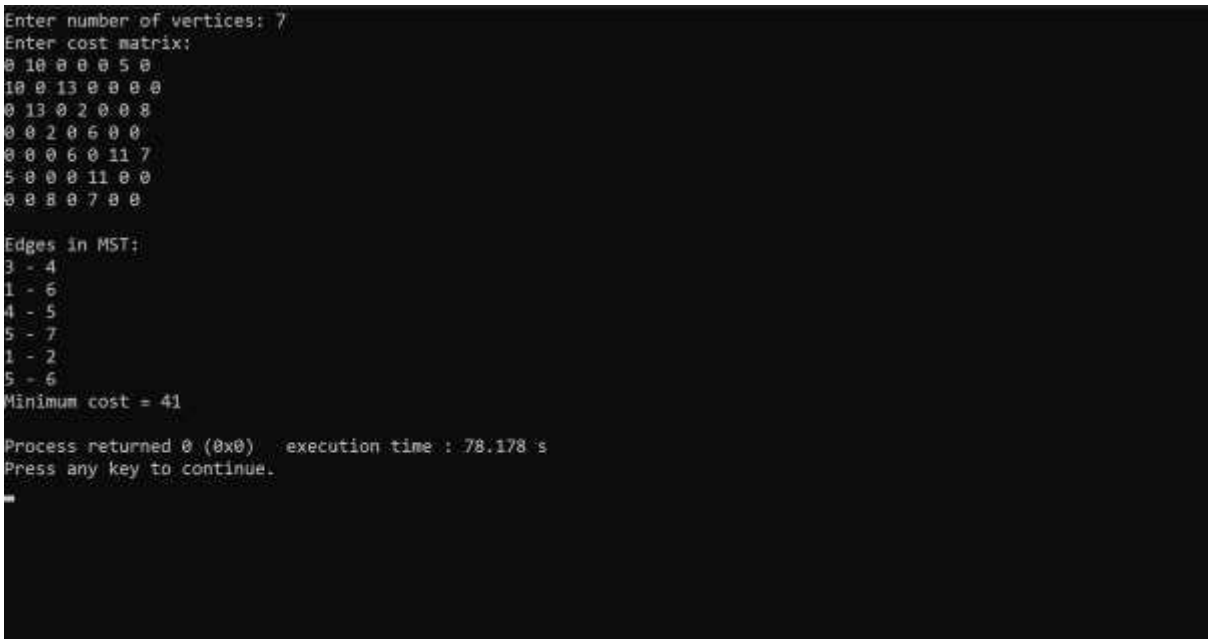
Lab Program 8(b): Kruskal's Algorithm

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>

#define MAX 10
#define INF 9999

int parent[MAX];

int find(int i)
{
   while (parent[i])
      i = parent[i];
   return i;
}

void union_set(int i, int j)
{
   parent[j] = i;
}

int main()
{
   int n, cost[MAX][MAX];
   int t[MAX][2];
   int i, j, u, v;
   int min, mincost = 0;
   int edge_count = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter cost matrix:\n");
   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= n; j++)
      {
         scanf("%d", &cost[i][j]);
         if (cost[i][j] == 0)
            cost[i][j] = INF;
      }
```

```
    }

    for (i = 1; i <= n; i++)
        parent[i] = 0;

    while (edge_count < n - 1)
    {
        min = INF;

        for (i = 1; i <= n; i++)
        {
            for (j = 1; j <= n; j++)
            {
                if (cost[i][j] < min)
                {
                    min = cost[i][j];
                    u = i;
                    v = j;
                }
            }
        }

        i = find(u);
        j = find(v);

        if (i != j)
        {
            edge_count++;
            t[edge_count][1] = u;
            t[edge_count][2] = v;
            mincost += min;
            union_set(i, j);
        }

        cost[u][v] = cost[v][u] = INF;
    }

    printf("\nEdges in MST:\n");
    for (i = 1; i <= edge_count; i++)
    {
        printf("%d - %d\n", t[i][1], t[i][2]);
    }

    printf("Minimum cost = %d\n", mincost);

    return 0;
}
```

Output:


Enter number of vertices: 7
Enter cost matrix:
0 10 0 0 0 5 0
10 0 13 0 0 0 0
0 13 0 2 0 0 8
0 0 2 0 6 0 0
0 0 0 6 0 11 7
5 0 0 0 11 0 0
0 0 8 0 7 0 0
Edges in MST:
3 - 4
1 - 6
4 - 5
5 - 7
1 - 2
5 - 6
Minimum cost = 41
Process returned 0 (0x0) execution time : 78.178 s
Press any key to continue.

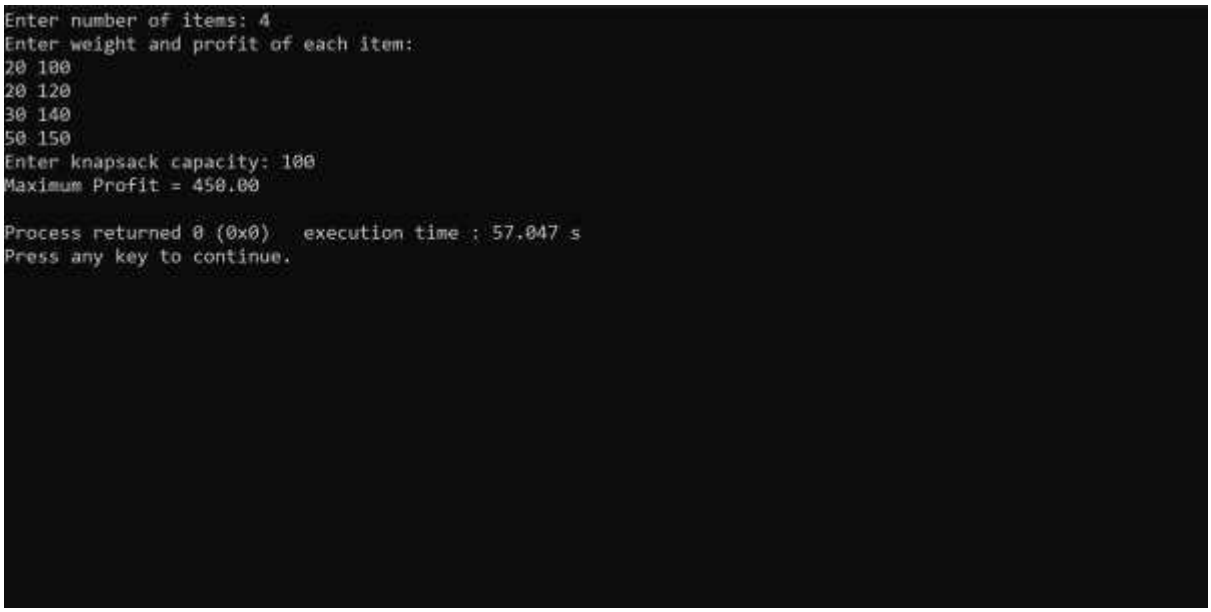
Lab Program 9: Fractional Knapsack

Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>

struct Item
{
   int weight;
   int profit;
   float ratio;
};

void sort(struct Item item[], int n)
{
   struct Item temp;

   for(int i = 0; i < n - 1; i++)
   {
      for(int j = 0; j < n - i - 1; j++)
      {
         if(item[j].ratio < item[j + 1].ratio)
         {
            temp = item[j];
            item[j] = item[j + 1];
            item[j + 1] = temp;
         }
      }
   }
}

int main()
{
   int n, capacity;
   float totalProfit = 0;

   printf("Enter number of items: ");
   scanf("%d", &n);

   struct Item item[n];

   printf("Enter weight and profit of each item:\n");
   for(int i = 0; i < n; i++)
   {
      scanf("%d%d", &item[i].weight, &item[i].profit);
      item[i].ratio = (float)item[i].profit / item[i].weight;
```

```
    }

    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);

    sort(item, n);

    for(int i = 0; i < n; i++)
    {
        if(capacity >= item[i].weight)
        {
            totalProfit += item[i].profit;
            capacity -= item[i].weight;
        }
        else
        {
            totalProfit += item[i].profit * ((float)capacity / item[i].weight);
            break;
        }
    }

    printf("Maximum Profit = %.2f\n", totalProfit);

    return 0;
}
```

Output:



Lab Program 10: Dijkstra's Algorithm

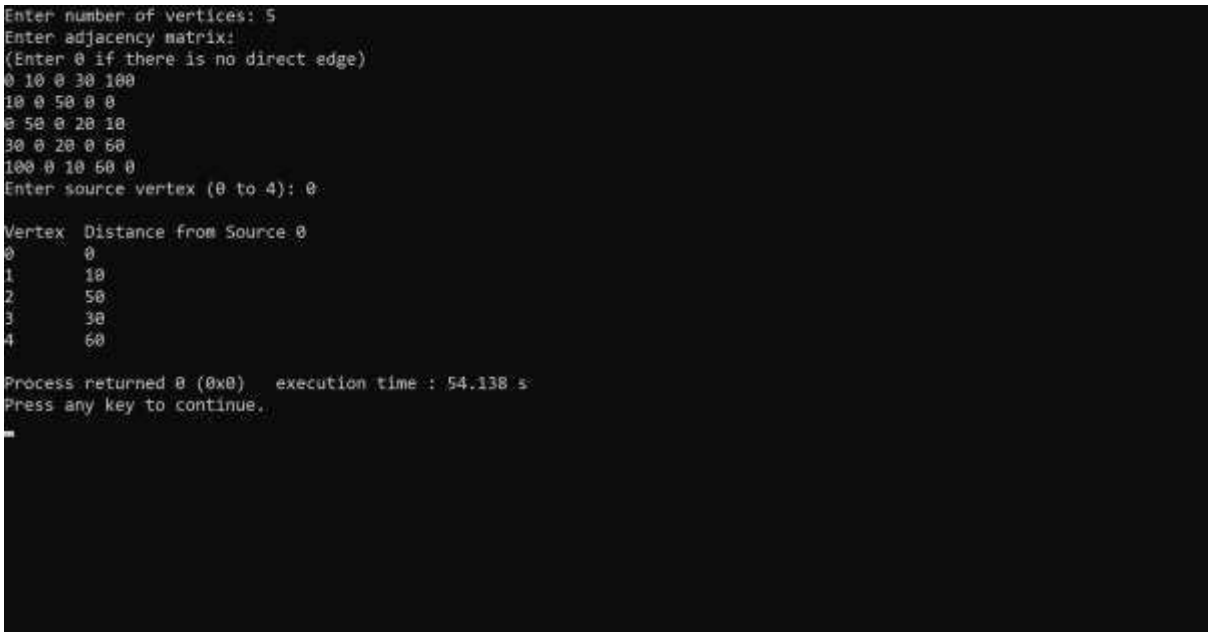
From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>

#define INF 9999

int minDistance(int dist[], int visited[], int n)
{
   int min = INF, min_index = -1;

   for(int v = 0; v < n; v++)
   {
      if(!visited[v] && dist[v] < min)
      {
         min = dist[v];
         min_index = v;
      }
   }

   return min_index;
}

void printSolution(int dist[], int n, int src)
{
   printf("\nVertex\tDistance from Source %d\n", src);

   for(int i = 0; i < n; i++)
   {
      printf("%d\t%d\n", i, dist[i]);
   }
}

void dijkstra(int graph[20][20], int n, int src)
{
   int dist[20], visited[20];

   for(int i = 0; i < n; i++)
   {
      dist[i] = INF;
      visited[i] = 0;
   }

   dist[src] = 0;
```

```
    for(int count = 0; count < n - 1; count++)
    {
        int u = minDistance(dist, visited, n);

        if(u == -1)
            break;

        visited[u] = 1;

        for(int v = 0; v < n; v++)
        {
            if(!visited[v] &&
               graph[u][v] &&
               dist[u] != INF &&
               dist[u] + graph[u][v] < dist[v])
            {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printSolution(dist, n, src);
}

int main()
{
    int n, graph[20][20], src;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    printf("(Enter 0 if there is no direct edge)\n");

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter source vertex (0 to %d): ", n - 1);
    scanf("%d", &src);

    dijkstra(graph, n, src);
```

```
    return 0;
}
```

Output:



Lab Program 11: N-Queens Problem

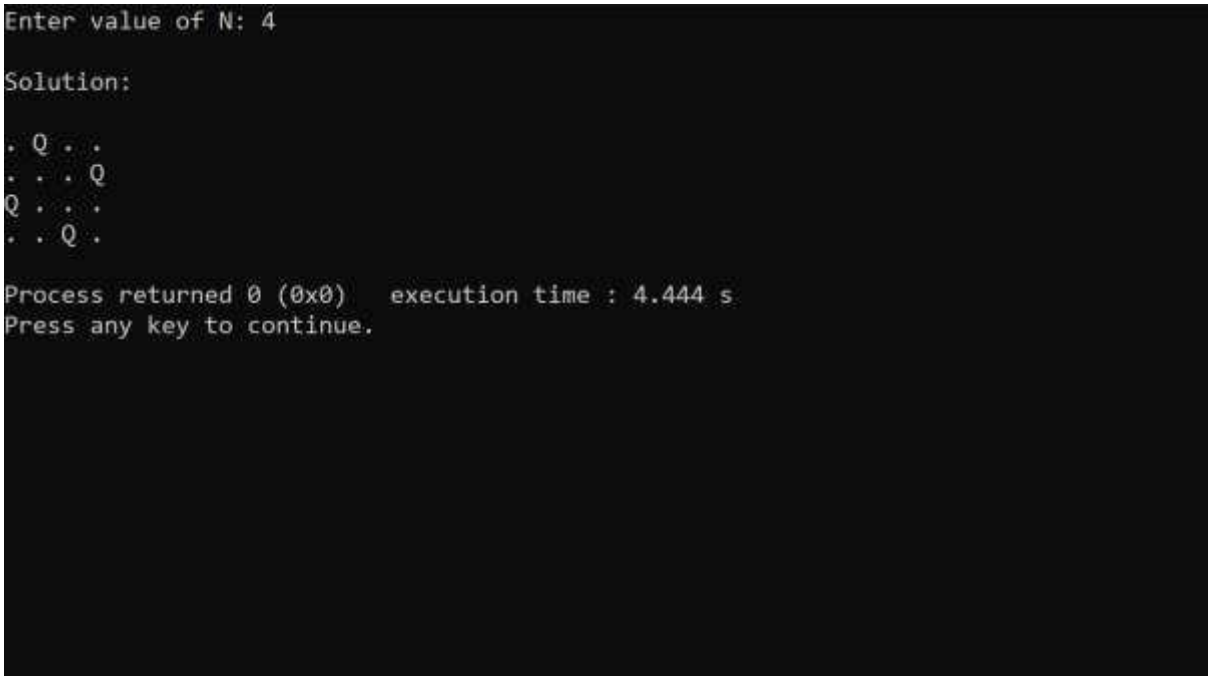
```
#include <stdio.h>

#define MAX 20

int board[MAX][MAX];
int N;

// Function to check whether position is safe
int isSafe(int row, int col)
{
   int i;

   // Check same column
   for(i = 0; i < row; i++)
   {
      if(board[i][col] == 1)
         return 0;
   }

   // Check upper left diagonal
   for(i = 1; row - i >= 0 && col - i >= 0; i++)
   {
      if(board[row - i][col - i] == 1)
         return 0;
   }

   // Check upper right diagonal
   for(i = 1; row - i >= 0 && col + i < N; i++)
   {
      if(board[row - i][col + i] == 1)
         return 0;
   }
```

```
    return 1;
}

// Backtracking function
int solveNQueens(int row)
{
    int col;

    // All queens placed
    if(row == N)
        return 1;

    // Try all columns
    for(col = 0; col < N; col++)
    {
        if(isSafe(row, col))
        {
            // Place queen
            board[row][col] = 1;

            // Recursive call
            if(solveNQueens(row + 1))
                return 1;

            // Backtrack
            board[row][col] = 0;
        }
    }

    return 0;
}

// Function to print board
```

```
void printBoard()
{
   int i, j;

   printf("\nSolution:\n\n");

   for(i = 0; i < N; i++)
   {
      for(j = 0; j < N; j++)
      {
         if(board[i][j] == 1)
            printf("Q ");
         else
            printf(". ");
      }

      printf("\n");
   }
}

int main()
{
   int i, j;

   printf("Enter value of N: ");
   scanf("%d", &N);

   // Initialize board with 0
   for(i = 0; i < N; i++)
   {
      for(j = 0; j < N; j++)
      {
         board[i][j] = 0;
      }
```

```
    }

    if(solveNQueens(0))
        printBoard();
    else
        printf("No solution exists.\n");

    return 0;
}
```

Output:



All Leetcode Problems:

Leetcode 207: Course Schedule

There is a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisite where prerequisites[i] = [a_i, b_i] indicates that you must take course b_i first if you want to take course a_i. For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1. Return true if you can finish all courses. Otherwise, return false.

Example 1:
Input: numCourses = 2, prerequisites = [[1,0]]
Output: true
Explanation: There is a total of 2 courses to take. To take course 1 you should have finished course 0. So, it is possible.

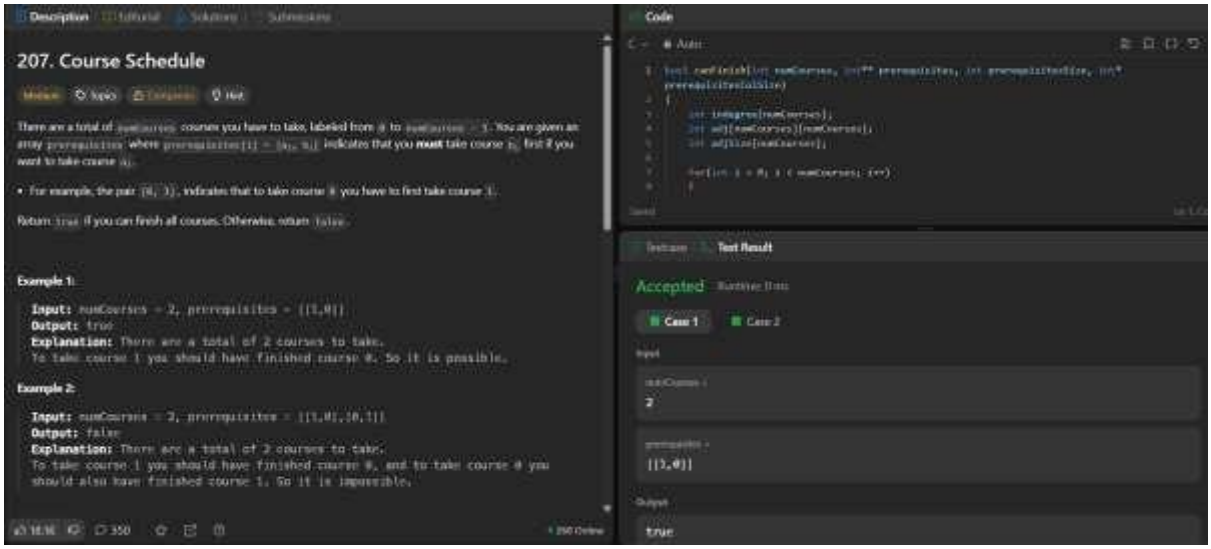
Example 2:
Input: numCourses = 2, prerequisites = [[1,0], [0,1]]
Output: false
Explanation: There is a total of 2 courses to take. To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So, it is impossible.

Solution:

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {

   int graph[2000][2000] = {0};

   int indegree[2000] = {0};

   int queue[2000];


   int front = 0, rear = -1;

   int i, count = 0;


   for(i = 0; i < prerequisitesSize; i++) {

      int a = prerequisites[i][0];

      int b = prerequisites[i][1];
```

```
        graph[b][a] = 1;
        indegree[a]++;
    }

    for(i = 0; i < numCourses; i++) {
        if(indegree[i] == 0) {
            queue[++rear] = i;
        }
    }

    while(front <= rear) {
        int u = queue[front++];
        count++;

        for(i = 0; i < numCourses; i++) {
            if(graph[u][i] == 1) {
                indegree[i]--;

                if(indegree[i] == 0) {
                    queue[++rear] = i;
                }
            }
        }
    }

    return count == numCourses;
}
```

Output:


207. Course Schedule
Code
Test Result
Accepted
Case 1
Case 2
2
[[1,0]]
true

Minimum swap:

You are given two integer arrays of the same length nums1 and nums2. In one operation, you are allowed to swap nums1[i] with nums2[i].

- For example, if nums1 = [1,2,3,8], and nums2 = [5,6,7,4], you can swap the element at i = 3 to obtain nums1 = [1,2,3,4] and nums2 = [5,6,7,8].

Return *the minimum number of needed operations to make* nums1 *and* nums2 ***strictly increasing***. The test cases are generated so that the given input always makes it possible.

An array arr is **strictly increasing** if and only if arr[0] < arr[1] < arr[2] < ... < arr[arr.length - 1].

Example 1:

Input: nums1 = [1,3,5,4], nums2 = [1,2,3,7]

Output: 1

Explanation:

Swap nums1[3] and nums2[3]. Then the sequences are:

nums1 = [1, 3, 5, 7] and nums2 = [1, 2, 3, 4]

which are both strictly increasing.

Example 2:

Input: nums1 = [0,3,5,8,9], nums2 = [2,1,4,6,9]

Output: 1

Constraints:

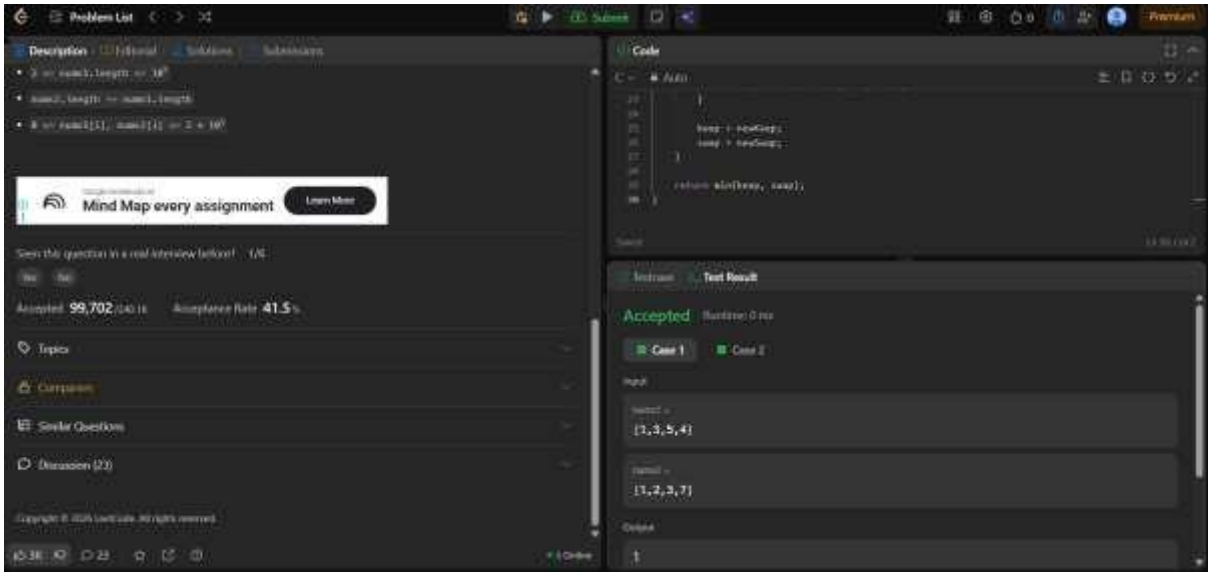
- 2 <= nums1.length <= 10^5
- nums2.length == nums1.length
- 0 <= nums1[i], nums2[i] <= 2 * 10^5

Program:

```
int min(int a, int b) {
   return a < b ? a : b;
}
```

```
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int keep = 0;   // no swap at current position
    int swap = 1;   // swap at current position

    for (int i = 1; i < nums1Size; i++) {
        int newKeep = 100000;
        int newSwap = 100000;

        // Case 1: no crossing
        if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1]) {
            newKeep = min(newKeep, keep);
            newSwap = min(newSwap, swap + 1);
        }

        // Case 2: crossing
        if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1]) {
            newKeep = min(newKeep, swap);
            newSwap = min(newSwap, keep + 1);
        }

        keep = newKeep;
        swap = newSwap;
    }

    return min(keep, swap);
}
```

Output:


Problem List
Description
Mind Map every assignment
Learn More
Accepted 99,702
Acceptance Rate 41.5%
Topics
Similar Questions
Discussion (23)
Code
Test Result
Accepted
Case 1
Case 2
[1,3,5,4]
[1,2,3,7]

Leetcode 1221: Split a String in Balanced Strings

Balanced strings are those that have an equal quantity of 'L' and 'R' characters. Given a balanced string s, split it into some number of substrings such that each substring is balanced. Return the maximum number of balanced strings you can obtain.

Example 1:
Input: s = "RLRRLLRLRL"
Output: 4
Explanation: s can be split into "RL", "RRLL", "RL", "RL", each substring contains same number of 'L' and 'R'.

```
int balancedStringSplit(char* s) {

   int bal = 0, count = 0;



   for (int i = 0; s[i] != '\0'; i++) {

      if (s[i] == 'L')

         bal++;

      else

         bal--;



       if (bal == 0)

          count++;

   }



   return count;

}
```

Output:


Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
s =
"RLRRLLRLRL"
Output
4
Expected
4

Leetcode 322: Coin Change

You are given an integer array coin representing coins of different denominations and an integer amount representing a total amount of money.
Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1.
You may assume that you have an infinite number of each kind of coin.

Example 1
Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1

Solution:

```
int coinChange(int* coins, int coinsSize, int amount) {
   int dp[amount + 1];


   for (int i = 0; i <= amount; i++)
      dp[i] = amount + 1;


   dp[0] = 0;


   for (int i = 1; i <= amount; i++) {
      for (int j = 0; j < coinsSize; j++) {
         if (coins[j] <= i) {
            int curr = 1 + dp[i - coins[j]];
            if (curr < dp[i])
               dp[i] = curr;
         }
```

```
        }
    }

    return (dp[amount] > amount) ? -1 : dp[amount];
}
```

Output:

Leetcode 316: Remove Duplicate Letters

Given a string s, remove duplicate letters so that every letter appears once and only once. You must make sure your result is the smallest in lexicographical order among all possible results.

Example 1:
Input: s = "bcabc"
Output: "abc"

Example 2:
Input: s = "cbacdcbc"
Output: "acdb"

Constraints:

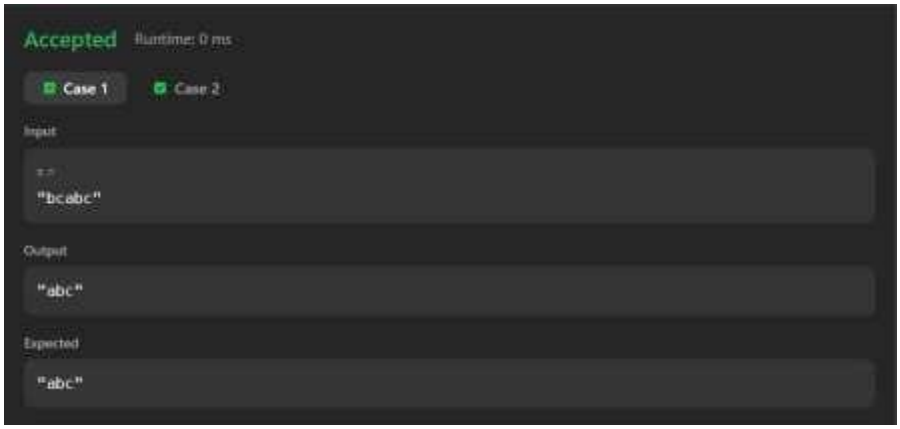
- $1 \leq s.length \leq 104$
- s consists of lowercase English letters.

Solution:

```
#include <string.h>

#include <stdbool.h>

#include <stdlib.h>


char* removeDuplicateLetters(char* s) {

   int last[26] = {0};

   bool used[26] = {false};


   int n = strlen(s);


   for (int i = 0; i < n; i++) {

      last[s[i] - 'a'] = i;

   }


   char *result = (char *)malloc((n + 1) * sizeof(char));

   int size = 0;
```

```
    for (int i = 0; i < n; i++) {
        char c = s[i];
        int idx = c - 'a';

        if (used[idx])
            continue;

        while (size > 0 &&
               result[size - 1] > c &&
               last[result[size - 1] - 'a'] > i) {
            used[result[size - 1] - 'a'] = false;
            size--;
        }

        result[size++] = c;
        used[idx] = true;
    }

    result[size] = '\0';
    return result;
}
```

Output:


Accepted
Runtime: 0 ms
Case 1
Case 2
Input
"bcabc"
Output
"abc"
Expected
"abc"